

KOI (KAIST Opensource Impact)

교육 기획서: AI 시대의 오픈소스 실전

과정 개요

- 과정명: AI 시대의 오픈소스 실전: 기여자에서 메인테이너까지
- 한 줄 소개: 대학생·대학원생이 AI 시대의 오픈소스 생태계를 이해하고 GitHub 협업, 라이선스, 기여 문화, AI 개발 도구를 실습해 6주 안에 실제 기여 또는 공개 프로젝트 운영 경험을 완성하는 실전형 과정
- 대상: KAIST 대학생·대학원생
- 기간: 2026-05-06부터 2026-06-04까지, 총 6회
- 시간: 매회 19:00-21:00
- 운영 형식: 강의, 데모, 짧은 실습, 팀별 멘토링
- 최종 약속: 6주 후 모든 팀이 하나의 공개 저장소를 운영하거나 외부 오픈소스 프로젝트에 실제 기여 흔적을 남긴다.

기획 배경

오픈소스 교육은 단순한 GitHub 사용법이나 저장소 공개 경험에 머물러서는 충분하지 않다. AI, 에이전트, 타입 언어가 개발 생태계의 변화를 만들고 있으며 GitHub 활동도 증가하고 있다는 문제의식을 제시한다. 따라서 이 과정은 학생들이 오픈소스를 사용하는 수준을 넘어 Issue, Branch, Commit, Pull Request, Review, LICENSE, CONTRIBUTING, README, AI 사용 로그까지 공개적으로 확인 가능한 결과물을 남기도록 설계한다.

이 과정에서 AI는 별도 주제가 아니라 매주 반복되는 작업 방식이다. 1회차에는 프로젝트 분석, 2회차에는 PR 요약과 이슈 정리, 3회차에는 라이선스 검토 질문, 4회차에는 기여·문서화·테스트 자동화, 5회차에는 멀티 에이전트 개발 흐름, 6회차에는 포트폴리오 정리에 활용한다. 모든 AI 활용은 사람이 검증한 근거와 함께 기록한다.

학습 목표

수료 시 학습자는 다음을 수행할 수 있어야 한다.

1. 오픈소스 생태계와 기여 문화를 설명하고, 프로젝트의 README, 이슈, PR, 릴리즈, 라이선스 구조를 읽어낼 수 있다.
2. GitHub 기반 협업 워크플로우를 사용해 Issue 생성, Branch 분리, Commit 작성, Pull Request 제출, Review 대응을 수행할 수 있다.
3. LICENSE, CONTRIBUTING, CODE_OF_CONDUCT, SECURITY, Issue Template, PR Template이 저장소 신뢰 구조에서 맡는 역할을 설명할 수 있다.
4. AI를 활용해 낯선 저장소 분석, 이슈 재현, PR 요약, 문서화, 테스트 작성, 리뷰 대응을 보조하되 사람이 검증해야 할 지점을 구분할 수 있다.
5. 팀 단위로 외부 오픈소스 기여를 시도하거나 자체 AI·오픈소스 프로젝트를 공개 저장소로 운영할 수 있다.
6. 포트폴리오와 커리어에 연결 가능한 공개 증거를 정리할 수 있다.

운영 구조

기본 운영 원칙

- 강의는 핵심 개념, 사례, 데모 중심으로 운영하며 매회 주제별 개념을 실제 저장소 화면과 연결한다.

- 실습은 20-30분의 짧은 작업 블록으로 운영하며 이슈 작성, PR 초안, 템플릿 추가, AI 검증 로그 작성처럼 제출 가능한 산출물을 만든다.
- 멘토링은 팀별 프로젝트와 기여 방향을 점검하는 시간으로 운영하며 매회 말 팀별 의사결정과 다음 액션을 확정한다.
- 공개 증거는 저장소와 외부 기여 기록을 중심으로 남긴다. 핵심 증거는 Issue, PR, Commit, Review, README, LICENSE, AI 사용 로그다.

팀 역할

- Maintainer: 저장소 방향, 이슈 우선순위, PR 병합 기준, 발표 흐름을 관리한다.
- Contributor: 기능 구현, 버그 수정, 테스트 추가, 외부 프로젝트 기여를 실행한다.
- Docs: README, CONTRIBUTING, 튜토리얼, 발표 자료, 포트폴리오를 문서화한다.
- AI/Automation: AI 사용 로그, 자동화 설정, 테스트·리뷰 보조, 인간 검증 체크리스트를 관리한다.

팀 프로젝트 트랙

A트랙: 외부 오픈소스 기여

기존 오픈소스 프로젝트에 실제 기여를 시도하는 트랙이다. 기존 코드 읽기와 커뮤니케이션을 경험하고 싶은 팀에 적합하다. 최소 증거는 외부 이슈, 재현 기록, PR 또는 문서·테스트 기여 흔적이다. README 개선, 번역, 예제 추가에서 기능 구현·CI 개선까지 난이도를 확장할 수 있다. PR이 병합되지 않아도 제출, 리뷰, 대응 기록을 인정한다.

B트랙: 자체 AI·오픈소스 운영

작은 오픈소스 프로젝트를 직접 만들고 공개 저장소를 운영하는 트랙이다. 문제 정의, 저장소 설계, 운영 문서화를 직접 경험하고 싶은 팀에 적합하다. 최소 증거는 공개 저장소, README, LICENSE,

Issue/PR/Commit 기록이다. 교육 자료, 개발자 도구, RAG 데모, MCP 예제, 캠퍼스 자동화 도구처럼 범위를 줄일 수 있다. 기능 규모보다 운영 구조와 공개 증거의 완성도를 우선한다.

회차별 산출물

1. 1회차, 2026-05-06: 프로젝트 또는 기여 후보 3개, 팀 구성, 트랙 후보
2. 2회차, 2026-05-07: 팀 저장소, 첫 Issue 목록, 첫 PR 또는 PR 계획, 역할 분담
3. 3회차, 2026-05-14: LICENSE, CONTRIBUTING.md 초안, README 개선, Issue/PR Template 또는 운영 규칙
4. 4회차, 2026-05-21: 외부 프로젝트 기여 PR 초안 또는 자체 프로젝트 기능 PR, AI 사용 로그, 테스트·문서·리뷰 체크리스트
5. 5회차, 2026-05-28: 기능 PR 1개, 테스트 또는 데모, 문서 업데이트, 멀티 에이전트 사용 기록, 최종 발표 초안
6. 6회차, 2026-06-04: 최종 발표, 공개 저장소 또는 외부 기여 흔적, 포트폴리오 정리, 다음 기여 계획

최종 제출물

- 공개 저장소 또는 외부 기여: 저장소 URL, 외부 Issue/PR URL, Commit 기록, Review 대응 기록
- 운영 문서: README, LICENSE, CONTRIBUTING, CODE_OF_CONDUCT 또는 SECURITY 중 필요한 항목
- 프로젝트 관리 기록: Issue 목록, 라벨, Milestone 또는 프로젝트 보드, PR Template
- AI 활용 기록: 사용 목적, 입력한 요청 요약, AI 결과, 사람이 검증한 항목, 수정 내역
- 발표 자료: 문제 정의, 선택한 트랙, 6주간의 Issue/PR/Commit, 라이선스와 운영 정책, 향후 로드맵
- 개인 회고: 본인의 기여, 배운 점, 다음 오픈소스 액션 플랜

평가 기준

- 오픈소스 이해도, 20%: 라이선스 선택 이유, 기여 문화 설명, 운영 파일의 역할 이해, 프로젝트 구조 분석을 인정 증거로 본다.
- GitHub 협업, 20%: Issue, Branch, Commit, Pull Request, Review 사용 기록과 커뮤니케이션 품질을 인정 증거로 본다.
- 실제 기여·운영, 25%: 외부 PR·이슈 기여 또는 자체 공개 저장소 운영 결과, 기능·문서·테스트 산출물을 인정 증거로 본다.
- AI 활용 능력, 20%: AI를 기여·리뷰·문서화·테스트에 적절히 활용하고 사람이 검증한 로그를 인정 증거로 본다.
- 발표·문서화, 15%: README, 발표 자료, 회고, 포트폴리오 연결성, 향후 로드맵을 인정 증거로 본다.

운영 리스크와 대응

- 학습자 간 Git/GitHub 숙련도 차이: 1-2회차에 작은 실습을 배치하고, 팀 내 역할을 분리해 입문자는 문서·이슈·재현부터 시작한다.
- 외부 PR이 6주 안에 병합되지 않음: 병합 여부보다 좋은 이슈 작성, 재현, PR 제출, 리뷰 대응 기록을 평가 증거로 인정한다.
- 라이선스·법률 내용을 과도하게 단정함: 교육 목적의 선택 기준과 확인 절차로 설명하고, 실제 법률 판단은 공식 문서와 전문가 확인이 필요하다고 명시한다.
- AI 생성 결과의 오류: 모든 회차에 인간 검증 항목을 포함하고, AI 사용 로그에 검증 내용과 수정 내역을 남긴다.
- 공개 산출물이 약함: 매회 산출물을 최종 발표 항목과 연결하고, Issue/PR/Commit/README 등 확인 가능한 증거를 우선한다.
- 자체 프로젝트 범위가 과도함: 1회차 후보 검토와 2회차 역할 분담에서 최소 기능, 문서, 운영 파일 중심으로 범위를 줄인다.

성공 기준

과정 종료 시 팀별로 다음 중 하나 이상이 공개적으로 확인되어야 한다.

- 외부 오픈소스 기여 흔적: 외부 프로젝트 Issue, PR, 리뷰 대화, Commit, 문서·테스트 기여 URL
- 자체 공개 저장소 운영: 공개 GitHub 저장소, README, LICENSE, CONTRIBUTING, Issue/PR 기록
- AI 활용과 검증 기록: AI 사용 로그, 검증 체크리스트, 사람이 수정한 근거
- 발표와 포트폴리오 연결: 최종 발표 자료, GitHub 프로필 또는 pinned repo 정리, 다음 기여 계획

커리큘럼: AI 시대의 오픈소스 실전

역량 맵

- 오픈소스 리터러시: 오픈소스 문화, 프로젝트 구조, 기여 유형, 메인테이너 관점을 이해한다. 확인 증거는 프로젝트 후보 분석, README·Issue·PR 분석이다.
- GitHub 협업: Issue, Branch, Commit, PR, Review 흐름으로 협업한다. 확인 증거는 팀 저장소, 첫 Issue, PR, 리뷰 대응이다.
- 운영·거버넌스: LICENSE, CONTRIBUTING, CODE_OF_CONDUCT, SECURITY, 템플릿으로 신뢰 구조를 만든다. 확인 증거는 운영 파일, 템플릿, 운영 규칙 문서다.
- AI 보조 개발: AI를 코드 이해, 이슈 분석, 문서화, 테스트, PR 작성, 리뷰 대응에 활용한다. 확인 증거는 AI 사용 로그와 사람이 검증한 체크리스트다.
- 공개 증거 구성: 포트폴리오에 남길 수 있는 공개 기록을 설계하고 발표한다. 확인 증거는 저장소 URL, PR/Commit/Review 기록, 발표 자료다.

전체 일정

1. 1회차, 2026-05-06 19:00-21:00

AI 시대의 오픈소스 왜 지금 다시 중요한가. 핵심 질문은 "AI 시대에 좋은 오픈소스 프로젝트는 무엇으로 판단하는가?"이다. GitHub Trending 또는 관심 프로젝트 3개를 분석하고, 프로젝트·기여 후보 3개, 팀 구성, 트랙 후보를 남긴다.

2. 2회차, 2026-05-07 19:00-21:00

GitHub 협업 워크플로우와 오픈소스 운영 기본기. 핵심 질문은 "좋은 기여는 어떻게 Issue와 PR로 기록되는가?"이다. Issue 작성, Branch 생성, PR 초안 작성을 실습하고, 팀 저장소, 첫 Issue 목록, 첫 PR 계획, 역할 분담을 남긴다.

3. **3회차, 2026-05-14 19:00-21:00**

오픈소스 라이선스, 거버넌스, 커뮤니티 운영. 핵심 질문은 "저장소의 신뢰는 어떤 문서와 규칙으로 만들어지는가?"이다. LICENSE·CONTRIBUTING·README를 점검하고, LICENSE, CONTRIBUTING 초안, README 개선, 템플릿을 남긴다.

4. **4회차, 2026-05-21 19:00-21:00**

AI를 활용한 오픈소스 기여와 운영 자동화. 핵심 질문은 "AI를 어디까지 맡기고 무엇을 사람이 검증해야 하는가?"이다. AI 기반 이슈 분석, PR 요약, 테스트·문서 체크를 실습하고, 기여 PR 초안 또는 기능 PR, AI 사용 로그, 검증 체크리스트를 남긴다.

5. **5회차, 2026-05-28 19:00-21:00**

AI 멀티 에이전트로 오픈소스 프로젝트 개발하기. 핵심 질문은 "역할이 나뉜 AI 작업 흐름은 개발 품질을 어떻게 높이는가?"이다. Planner-Coder-Reviewer-Tester-Documenter 흐름을 적용하고, 기능 PR, 테스트 또는 데모, 문서 업데이트, 발표 초안을 남긴다.

6. **6회차, 2026-06-04 19:00-21:00**

프로젝트 발표, 오픈소스 커리어, 다음 기여로 이어가기. 핵심 질문은 "이 경험을 공개적으로 검증 가능한 실력의 흔적으로 어떻게 정리하는가?"이다. 발표 리허설과 포트폴리오 증거 점검을 진행하고, 최종 발표, 공개 저장소·기여 흔적, 다음 기여 계획을 남긴다.

회차별 상세

1회차: AI 시대의 오픈소스: 왜 지금 다시 중요한가

핵심 메시지는 오픈소스가 코드 공개를 넘어 AI 시대의 학습, 협업, 커리어, 제품 확산 방식이라는 점이다.

- 학습 목표: 오픈소스 프로젝트의 공개 증거를 읽고, 좋은 후보를 비교하며, 팀 트랙 선택 기준을 설명한다.
- 주요 내용: 과정 목표, 최종 발표 기준, 오픈소스 변화, GitHub 트렌드 읽기, AI와 오픈소스, 최근 인기 프로젝트 분석
- 실습: README, Issue, PR, LICENSE, 최근 Commit, 기여 가이드 기준으로 프로젝트 3개를 비교한다.

- 멘토링: 팀 구성, A/B트랙 후보 선택, 후보별 리스크와 다음 조사 액션 확정
- 과제: 후보 3개 분석과 팀 역할 임시 배정안을 제출한다.

2회차: GitHub 협업 워크플로우와 오픈소스 운영 기본기

핵심 메시지는 오픈소스의 기본 언어가 Issue, Branch, Commit, Pull Request, Review라는 점이다.

- 학습 목표: GitHub Flow를 설명하고, 좋은 Issue와 PR 초안을 작성하며, 팀 저장소 운영 역할을 나눈다.
- 주요 내용: Repository, Fork, Clone, Branch, Commit, Push, GitHub Flow, Issue 운영, PR 작성법, 리뷰 문화, 운영 파일
- 실습: 팀 저장소에 첫 Issue를 만들고, 작은 변경 브랜치와 PR 초안을 작성한다.
- 멘토링: 트랙 확정, 역할 분담, 첫 번째 Issue 목록과 PR 계획 점검
- 과제: 팀 저장소 URL, Issue 목록, 첫 PR 링크 또는 PR 초안을 제출한다.

3회차: 오픈소스 라이선스, 거버넌스, 커뮤니티 운영

핵심 메시지는 오픈소스가 공개가 아니라 사용·수정·배포·기여를 허용하는 규칙과 신뢰 구조라는 점이다.

- 학습 목표: 주요 라이선스 차이를 비교하고, 저장소 운영 파일의 역할을 설명하며, 팀 저장소 신뢰 구조를 보강한다.
- 주요 내용: 오픈소스 정의, MIT·Apache-2.0·GPL·AGPL·LGPL·MPL, 라이선스 선택 기준, AI와 라이선스, 커뮤니티 운영, 메인테이너 역할
- 실습: 팀 저장소에 LICENSE와 CONTRIBUTING 초안을 추가하고 README 구조를 개선한다.
- 멘토링: 라이선스 선택 근거, 기여 규칙, 이슈·PR 템플릿, 보안 연락 방식 점검
- 과제: 운영 문서 PR과 라이선스 선택 메모를 제출한다.

4회차: AI를 활용한 오픈소스 기여와 운영 자동화

핵심 메시지는 AI가 코드를 대신 쓰는 도구가 아니라 오픈소스 기여의 진입장벽을 낮추는 기여 보조 시스템이라는 점이다.

- 학습 목표: AI로 저장소 구조를 분석하고, 이슈 원인을 추정하며, PR·문서·테스트 초안을 만든 뒤 사람이 검증할 수 있다.
- 주요 내용: AI 기반 코드 이해, 이슈 분석, PR 작성, 문서화, 운영 자동화, RAG·MCP·A2A·에이전트 워크플로우 개요
- 실습: 실제 이슈나 팀 기능을 대상으로 AI 분석 로그와 PR 초안을 만든다.
- 멘토링: 기여 PR 초안, 테스트·문서·리뷰 체크리스트, AI 사용 로그의 검증 수준 점검
- 과제: PR 초안 또는 기능 PR, AI 사용 로그, 검증 체크리스트를 제출한다.

5회차: AI 멀티 에이전트로 오픈소스 프로젝트 개발하기

핵심 메시지는 AI를 잘 쓰는 개발자가 하나의 챗봇에게 질문하는 사람이 아니라 역할이 나뉜 AI 작업 흐름을 설계하는 사람이라는 점이다.

- 학습 목표: Planner, Coder, Reviewer, Tester, Documenter 역할을 나눠 작은 기능 개발 흐름을 설계하고 검증한다.
- 주요 내용: 멀티 에이전트 개념, 이슈→설계→구현→테스트→문서→PR 흐름, 역할 분리, hallucination·보안·라이선스·의존성 검증, 자동화 예시
- 실습: 기능 PR 또는 문서·테스트 PR을 멀티 에이전트 방식으로 추진하고 사람 검토 결과를 기록한다.
- 멘토링: 기능 PR, 테스트 또는 데모, 문서 업데이트, 발표 초안 점검
- 과제: PR 링크, 데모 또는 테스트 증거, 멀티 에이전트 사용 기록, 발표 초안을 제출한다.

6회차: 프로젝트 발표, 오픈소스 커리어, 다음 기여로 이어가기

핵심 메시지는 오픈소스 경험이 수업 결과물이 아니라 공개적으로 검증 가능한 실력의 흔적이라는 점이다.

- 학습 목표: 6주간의 공개 증거를 발표하고, 포트폴리오로 정리하며, 다음 기여 액션을 설계한다.
- 주요 내용: 결과 발표, GitHub 프로필, pinned repo, 커리어 연결, 좋은 기여자의 태도, 메인테이너 성장, 개인 액션 플랜

- 실습: 발표 리허설과 공개 증거 체크리스트 점검
- 멘토링: 최종 발표 보완, README·로드맵·회고 정리, 다음 기여 계획
- 과제: 최종 발표 자료와 개인 회고를 제출한다.

프로젝트 트랙 운영

A트랙: 외부 오픈소스 기여

1. 후보 선정: 관심 프로젝트 3개를 비교하고 기여 가능성을 판단한다. 최소 증거는 후보 분석이다.
2. 기여 범위 확정: README 개선, 번역, 예제 추가, 이슈 재현, 테스트 추가, 작은 버그 수정 중 하나를 선택한다. 최소 증거는 선택 근거와 담당자다.
3. 커뮤니케이션: Issue나 Discussion에서 문제 정의와 기여 의사를 명확히 쓴다. 최소 증거는 외부 Issue/Discussion URL이다.
4. PR 제출: 작은 단위로 변경하고 테스트·문서 증거를 붙인다. 최소 증거는 PR URL 또는 초안이다.
5. 리뷰 대응: 메인테이너 피드백을 정리하고 수정 내역을 남긴다. 최소 증거는 Review 대화와 후속 Commit이다.

외부 PR이 병합되지 않아도 재현, 이슈 작성, PR 제출, 리뷰 대응의 품질을 평가한다.

B트랙: 자체 AI·오픈소스 운영

1. 문제 정의: 캠퍼스, 연구실, 동아리, 학습, 개발자 도구 등 작은 문제를 정한다. 최소 증거는 문제 정의와 사용자다.
2. 저장소 설계: README, LICENSE, Issue, Project 구조를 만든다. 최소 증거는 공개 저장소 URL이다.
3. 최소 기능 구현: RAG 데모, MCP 서버 예제, 프롬프트 관리 도구, Issue 정리 봇 등 작은 기능을 구현한다. 최소 증거는 Commit과 PR이다.
4. 운영 문서화: CONTRIBUTING, 템플릿, 로드맵, 보안·문의 방식을 정리한다. 최소 증거는 운영 파일이다.

5. 공개 발표: 문제, 기능, 운영 정책, AI 검증, 다음 로드맵을 설명한다. 최소 증거는 발표 자료와 회고다.

기능 규모보다 공개 저장소 운영 방식과 증거의 완성도를 우선한다.

평가 기준

- 오픈소스 이해도, 20%: 우수한 결과물은 라이선스, 기여 문화, 운영 구조를 프로젝트 사례와 연결해 설명한다. 보완이 필요한 결과물은 용어 설명은 가능하지만 자기 프로젝트 적용 근거가 약하다.
- GitHub 협업, 20%: 우수한 결과물은 Issue, PR, Review, Branch가 작은 단위로 정리되어 있고 대화 기록이 명확하다. 보완이 필요한 결과물은 Commit은 있으나 Issue/PR 흐름이나 리뷰 대응이 부족하다.
- 실제 기여·운영, 25%: 우수한 결과물은 외부 PR 또는 자체 저장소가 실제 사용 가능한 공개 증거를 갖춘다. 보완이 필요한 결과물은 산출물이 로컬 작업이나 발표 설명에만 머문다.
- AI 활용 능력, 20%: 우수한 결과물은 AI 사용 목적, 결과, 인간 검증, 수정 내역이 구체적으로 기록되어 있다. 보완이 필요한 결과물은 AI를 썼다는 언급은 있으나 검증 근거가 없다.
- 발표·문서화, 15%: 우수한 결과물은 README, 발표, 회고, 로드맵이 일관되고 포트폴리오로 연결 가능하다. 보완이 필요한 결과물은 발표는 가능하지만 문서와 공개 증거가 흩어져 있다.

운영 준비사항

- GitHub 데모: Fork, Branch, PR, Review, Issue Template을 보여줄 수 있는 샘플 저장소
- 프로젝트 사례: README, Issue, PR, Release, LICENSE가 잘 정리된 오픈소스 사례와 보완이 필요한 사례
- 라이선스 예시: MIT, Apache-2.0, GPL 계열의 차이를 설명할 수 있는 교육용 비교 자료
- AI 도구 경계: AI가 도와줄 수 있는 일과 사람이 검증해야 하는 일을 구분한 체크리스트
- 멘토링 기록: 팀별 트랙, 산출물, 위험, 다음 액션을 누적 관리할 템플릿
- 발표 체크: 최종 발표 항목과 공개 증거 URL을 확인할 루브릭

AI 시대의 오픈소스: 왜 지금 다시 중요한가

- 일시: 2026-05-06 수 19:00-21:00
- 주제: AI 시대의 오픈소스와 6주 과정 오리엔테이션
- 핵심 메시지: 오픈소스는 코드 공개만이 아니라 AI 시대의 학습, 협업, 커리어, 제품 확산 방식이다.
- 목표 산출물: 개인 GitHub 프로필 README 초안, 후보 프로젝트 3개, 팀 구성, 팀 운영 약속 초안, AI 후보 분석 로그
- 트랙: 전원 공통으로 시작하되 후보 정리 시 외부 오픈소스 기여와 자체 공개 저장소 운영 가능성을 모두 남긴다.

학습 목표

수강생은 이 회차 후 다음을 수행할 수 있어야 한다.

1. AI 시대에 오픈소스가 학습, 협업, 커리어, 제품 확산 방식으로 중요해진 이유를 설명한다.
2. GitHub 저장소를 README, Issue, Pull Request, Release, LICENSE 관점에서 살펴본다.
3. 6주 동안 따라갈 후보 프로젝트 3개를 정리한다.
4. 팀 구성과 팀 운영 약속 초안을 만든다.
5. AI가 제안한 후보 분석 결과를 사람이 검증하는 방식으로 첫 AI 사용 로그를 남긴다.

수업 전 준비

수강생은 GitHub 계정을 준비하고, AI 도구 하나에 로그인해 둔다. 최근 사용한 오픈소스 패키지 이름 하나를 메모해 온다. 관심 분야도 하나 이상 정한다. 예시는 웹, AI, 데이터, 개발자 도구, 교육, 캠퍼스 문제 해결이다.

가능하면 Git이 설치되어 있는지 확인한다.

None

```
git --version
```

Git 사용 경험이 없어도 수업에는 참여할 수 있다. 이 회차에서는 저장소를 읽고 후보를 고르는 것이 우선이다.

수업 흐름

1. 최근 사용한 패키지를 떠올리며 오픈소스를 이미 쓰고 있다는 사실을 확인한다.
2. 6주 과정의 최종 약속과 평가 기준을 공유한다.
3. AI 시대 오픈소스 생태계의 변화를 간단히 다룬다.
4. 관심 저장소를 5개 관점으로 분석한다.
5. AI로 후보 저장소를 요약하게 하고 사람이 검증한다.
6. 팀을 구성하고 팀 운영 약속 초안을 만든다.
7. 개인 회고와 다음 회차 준비 항목을 정리한다.

핵심 내용

이 과정의 중심 약속은 6주 후 모든 팀이 공개 저장소 하나를 운영하거나 외부 오픈소스 프로젝트에 실제 기여 흔적을 남기는 것이다. 발표만 잘하는 과정이 아니라 GitHub에 남는 공개 증거를 만드는 과정이다.

오픈소스 프로젝트를 볼 때 별 수만 보지 않는다. README가 친절한지, Issue가 관리되는지, Pull Request가 리뷰되는지, Release가 이어지는지, LICENSE가 명확한지 함께 본다. 이 다섯 가지가 이후 기여 가능성을 판단하는 출발점이다.

AI 도구는 후보 분석을 빠르게 도울 수 있다. 그러나 AI가 추천한 프로젝트나 기여 아이디어를 그대로 확정하지 않는다. 최근 활동, 라이선스, 기여 가이드, 이슈 난이도는 사람이 직접 확인해야 한다.

활동

오픈소스 사용 자각

각자 최근에 설치하거나 사용한 패키지 이름을 적는다. 예시는 `react`, `requests`, `pandas`, `pytorch`, `axios` 같은 라이브러리다. 그 패키지를 누가 만들었는지, 왜 만들었는지, 어떤 라이선스로 공개했는지 생각해 본다.

이 활동의 목적은 정답을 맞히는 것이 아니다. 이미 오픈소스를 쓰고 있었지만 그 출처와 규칙을 잘 모르고 있었다는 점을 확인하는 것이다.

저장소 5개 관점 분석

관심 분야의 GitHub 저장소를 고르고 다음 질문에 답한다.

- README만 보고 프로젝트 목적과 사용법을 이해할 수 있는가?
- 열린 Issue가 관리되고 있는가?
- 최근 Pull Request가 리뷰되고 병합되는가?
- Release 또는 버전 관리 흔적이 있는가?
- LICENSE가 명시되어 있는가?
- 6주 안에 작은 기여를 시도할 수 있는가?

팀은 후보 3개를 고른다. 각 후보에는 저장소 URL, 선택 이유, 예상 기여 방식, 주요 리스크, 다음 조사 항목을 적는다.

AI 후보 분석

후보 저장소 하나를 골라 AI에게 요약을 요청한다.

None

이 GitHub 저장소를 대학생 오픈소스 기여 수업 관점에서 분석해줘.
출력은 한 줄 요약, 첫 기여 후보 3개, 6주 안에 가능한 이유, 사람이 직접 확인해야 할 리스크로 나눠줘.
확실하지 않은 내용은 추정이라고 표시해줘.

AI 결과를 받은 뒤 사람이 다음을 확인한다.

- 실제 최근 Commit 날짜
- LICENSE 존재와 종류
- Issue와 Pull Request의 최근 관리 상태
- AI가 존재하지 않는 기능, 파일, 정책을 말하지 않았는지 여부

팀 운영 약속 초안

팀은 다음 내용을 한 페이지로 정리한다.

- 팀명
- 한 줄 비전

- 잠정 역할: Maintainer, Contributor, Docs, AI-Automation
- 정기 회의 방식
- 의사결정 방식
- 연락 두절이나 역할 미수행이 생겼을 때의 처리 방식
- 후보 프로젝트 3개

이 문서는 2회차 이후 팀 저장소의 [docs/team-constitution.md](#)로 옮길 수 있다.

멘토링

멘토링에서는 후보 3개와 팀 운영 약속을 점검한다. 팀이 지나치게 큰 프로젝트를 고른 경우 작은 문서 기여, 재현, 테스트 추가, 예제 보강 같은 방향으로 범위를 줄인다.

확인할 질문은 다음과 같다.

- 후보 3개가 실제 URL과 선택 이유를 갖추었는가?
- 최소 1개 후보는 6주 안에 작은 기여가 가능한가?
- 팀 운영 약속이 잠정적으로라도 정리되었는가?
- 팀 안에 GitHub 경험이 부족하다면 누가 어떤 방식으로 보완할 것인가?
- AI 후보 분석에서 사람이 검증한 항목이 남아 있는가?

AI 활용

AI는 README 요약, 저장소 구조 설명, 첫 기여 후보 추천, 후보 비교 초안 작성에 사용할 수 있다. 사람이 확인해야 할 항목은 최근 활동, 라이선스, 기여 가이드, 이슈 난이도, 팀 역량과 일정이다.

AI 사용 로그는 다음 항목을 문장으로 남긴다.

- 어떤 후보를 분석했는가
- AI에게 무엇을 요청했는가
- AI가 무엇을 제안했는가
- 사람이 무엇을 확인했는가
- 최종 후보 선정에 어떻게 반영했는가

과제

팀 과제는 후보 프로젝트 3개 정리와 팀 운영 약속 초안 작성이다. 개인 과제는 GitHub 프로필 README 초안 한 단락과 AI 후보 분석 로그 한 건이다.

제출 증거는 후보 저장소 URL, 후보 비교 문서, 팀 운영 약속 문서, 개인 프로필 초안, AI 사용 로그다.

참고 출처

- GitHub Octoverse 2025:
<https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/>
- GitHub Trending: <https://github.com/trending>
- GitHub Profile README:
<https://docs.github.com/en/account-and-profile/how-tos/profile-customization/managing-your-profile-readme>

GitHub 협업 워크플로우와 오픈소스 운영

- 일시: 2026-05-07 목 19:00-21:00
- 주제: GitHub Flow, Issue, Branch, Commit, Pull Request, Review
- 핵심 메시지: 오픈소스의 기본 언어는 코드가 아니라 Issue, Branch, Commit, Pull Request, Review이다.
- 목표 산출물: 팀 저장소, 라벨 붙은 Issue 5개, 첫 Pull Request 1개, 트랙 선언, 역할 분담, AI로 다듬은 Pull Request 본문

학습 목표

수강생은 이 회차 후 다음을 수행할 수 있어야 한다.

1. GitHub Flow의 기본 흐름을 자기 손으로 1회 실행한다.
2. 좋은 Issue와 좋은 Pull Request에 필요한 정보를 작성한다.
3. 팀 저장소를 만들고 첫 공개 협업 기록을 남긴다.
4. 외부 오픈소스 기여 또는 자체 공개 저장소 운영 중 하나를 잠정 선택한다.
5. AI가 다듬은 Pull Request 본문을 실제 변경 내용과 대조해 검증한다.

수업 전 준비

1회차에서 만든 후보 프로젝트 3개와 팀 운영 약속 초안을 준비한다. GitHub에 로그인하고, 가능하면 Git 사용자 정보를 설정한다.

None

```
git config --global user.name "Your Name"  
git config --global user.email "your-github-email@example.com"
```

GitHub CLI가 있으면 좋지만 필수는 아니다. 웹 UI만으로도 수업을 진행할 수 있다.

수업 흐름

1. 지난 회차의 후보 프로젝트를 팀 저장소 작업으로 연결한다.
2. 활발한 오픈소스 저장소의 Issue에서 자주 보이는 단어를 찾아본다.
3. GitHub Flow의 흐름을 확인한다.
4. 팀 저장소를 만들고 첫 Issue와 Pull Request를 만든다.
5. 좋은 Issue와 Pull Request의 구성 요소를 정리한다.
6. AI로 Pull Request 본문을 다듬고 사람이 검증한다.
7. 팀별 트랙과 역할을 확정한다.

핵심 내용

GitHub Flow는 작은 변경을 안전하게 협업하기 위한 흐름이다. main에서 새 Branch를 만들고, 작은 변경을 Commit하고, Pull Request를 열고, Review를 거쳐 Merge한다. 이 과정이 남기는 기록이 오픈소스 협업의 핵심 증거다.

좋은 Issue는 제목만 있는 요청이 아니다. 문제, 맥락, 재현 또는 예시, 완료 조건, 라벨이 있어야 한다. 좋은 Pull Request는 무엇을 바꿨는지뿐 아니라 왜 바꿨고 어떻게 확인했는지를 포함한다.

2회차의 목표는 완벽한 코드를 만드는 것이 아니다. 팀 저장소가 살아 있고, Issue와 Pull Request가 실제로 남아 있으며, 팀이 어떤 트랙으로 갈지 말할 수 있는 상태를 만드는 것이다.

활동

Issue 단어 찾기

활발한 오픈소스 저장소의 Issues 화면을 보고 자주 등장하는 단어를 찾는다. 예시는 **repro**, **feature request**, **good first issue**, **docs**, **help wanted**다.

이 단어들은 협업에서 중요한 의미를 가진다. 재현 가능한 버그만 고칠 수 있고, 기능 제안은 코드보다 토론이 먼저이며, 문서 기여는 좋은 첫 기여가 될 수 있다.

GitHub Flow 실행

팀 저장소에서 작은 변경을 하나 만든다. README에 팀 소개 한 줄을 추가하는 정도면 충분하다.

None

```
git switch -c docs/add-team-note
: README.md를 편집한다.
git add README.md
git commit -m "docs: add team note"
git push -u origin docs/add-team-note
```

이후 GitHub에서 Pull Request를 만들고 팀원 한 명이 Review를 남긴다. 가능하면 Merge까지 진행한다.

Issue 5개 작성

팀 저장소에 Issue 5개 이상을 만든다. 유형은 기능, 문서, 테스트, 버그, 운영, AI 사용 로그 중에서 선택한다.

각 Issue는 다음 내용을 포함한다.

- 무엇을 하려는가
- 왜 필요한가
- 완료 조건은 무엇인가
- 누가 맡을 것인가
- 어떤 라벨을 붙일 것인가

Pull Request 본문 작성

Pull Request 본문에는 다음 내용을 문장으로 쓴다.

- 무엇을 바꿨는가
- 왜 바꿨는가
- 어떻게 확인했는가
- 어떤 Issue와 연결되는가
- 리뷰어가 특히 봐야 할 점은 무엇인가

AI로 Pull Request 본문 다듬기

다음 프롬프트를 사용해 본문 초안을 만들 수 있다.

None

다음 Pull Request 변경 내용과 관련 Issue를 바탕으로 Pull Request 본문을 작성해줘.

포함할 내용은 무엇을 바꿨는지, 왜 바꿨는지, 어떻게 확인했는지, 관련 Issue, 리뷰어가 볼 점이야.

실제 변경에 없는 내용은 쓰지 마.

AI 결과를 그대로 쓰지 않는다. 실제 diff와 맞는지, 없는 테스트가 들어가지 않았는지, Issue 번호가 맞는지 사람이 확인한다.

트랙 선언

팀은 한 줄로 트랙을 선언한다.

A트랙 선언 예시는 다음과 같다.

None

우리 팀은 A트랙이다. 외부 저장소의 문서 또는 테스트 기여를 6주 안에 시도한다.

B트랙 선언 예시는 다음과 같다.

None

우리 팀은 B트랙이다. 캠퍼스 문제를 해결하는 작은 공개 저장소를 만들고 운영한다.

외부 응답이 지연될 가능성이 큰 팀은 A트랙을 우선하되 B트랙 보조 산출물을 준비한다고 선언할 수 있다.

멘토링

멘토링에서는 팀 저장소, Issue, Pull Request, 트랙 선언, 역할 분담을 확인한다.

확인할 질문은 다음과 같다.

- 팀 저장소가 실제로 열리는가?
- Issue 5개가 제목만 있는 상태가 아닌가?
- Pull Request가 실제 변경을 포함하는가?
- Review 또는 코멘트 기록이 있는가?

- 트랙 선언이 구체적인가?
- Maintainer, Contributor, Docs, AI-Automation 역할이 정해졌는가?
- AI 사용 로그가 남아 있는가?

AI 활용

AI는 Issue 제목 개선, Pull Request 본문 초안, Commit 메시지 제안, README 구조 제안에 사용할 수 있다. 사람이 확인해야 할 것은 실제 변경 내용, 테스트 결과, 링크, 과장 표현, 팀의 실제 합의다.

AI 사용 로그에는 다음을 남긴다.

- 어떤 Pull Request 본문을 다듬었는가
- AI가 어떤 문장을 제안했는가
- 사람이 어떤 문장을 삭제하거나 수정했는가
- 최종 Pull Request 본문에 무엇이 반영되었는가

과제

팀 과제는 팀 저장소, Issue 5개 이상, 첫 Pull Request, 트랙 선언, 역할 분담 문서를 제출하는 것이다. 개인 과제는 자신이 작성하거나 리뷰한 Issue 또는 Pull Request 링크와 회고를 남기는 것이다.

제출 증거는 저장소 URL, Issue URL, Pull Request URL, 역할 분담 문서, AI 사용 로그다.

참고 출처

- GitHub Flow: <https://docs.github.com/en/get-started/using-github/github-flow>
- About Pull Requests:
<https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests>

- About Issues:

<https://docs.github.com/en/issues/tracking-your-work-with-issues/learning-about-issues/about-issues>

오픈소스 라이선스, 거버넌스, 커뮤니티 운영

- 일시: 2026-05-14 목 19:00-21:00
- 주제: LICENSE, CONTRIBUTING, CODE_OF_CONDUCT, SECURITY, Issue Template, Pull Request Template
- 핵심 메시지: 오픈소스는 공개가 아니라 사용, 수정, 배포, 기여를 허용하는 규칙과 신뢰 구조이다.
- 목표 산출물: 운영 신뢰 구조 문서, 라이선스 선택 메모, AI 라이선스 리스크 검토 로그, CONTRIBUTING의 라이선스 정책 섹션

학습 목표

수강생은 이 회차 후 다음을 수행할 수 있어야 한다.

1. 주요 오픈소스 라이선스의 차이를 사용 시나리오 중심으로 설명한다.
2. LICENSE, README, CONTRIBUTING, CODE_OF_CONDUCT, SECURITY, Issue Template, Pull Request Template이 저장소 신뢰에 기여하는 방식을 설명한다.
3. AI 프로젝트에서 코드, 데이터, 모델, weights, 문서의 공개 범위가 다를 수 있음을 설명한다.
4. 팀 저장소에 운영 문서를 Pull Request로 추가한다.
5. AI가 제안한 라이선스 리스크를 공식 출처와 팀 운영 정책으로 검증한다.

수업 전 준비

수강생은 2회차 팀 저장소와 첫 Pull Request, Issue 목록, 트랙 선언문을 준비한다. 팀 저장소에 LICENSE가 있는지 확인하고, 없다면 어떤 라이선스를 선택할지 생각해 온다.

외부 기여를 시도하는 팀은 대상 저장소의 LICENSE와 CONTRIBUTING을 읽어 온다. 자체 프로젝트를 운영하는 팀은 프로젝트가 라이브러리, CLI, 웹 서비스, 문서, 데이터 중 어떤 형태에 가까운지 정리한다.

수업 흐름

1. 지난 회차의 저장소가 외부인이 신뢰할 수 있는 상태인지 확인한다.
2. "오픈"이라는 표현과 오픈소스 라이선스의 차이를 다룬다.
3. 주요 라이선스의 차이를 사용 시나리오로 정리한다.
4. 팀 저장소에 운영 신뢰 구조 문서를 추가한다.
5. AI 시대의 라이선스 쟁점을 다룬다.
6. AI에게 팀 프로젝트의 라이선스 리스크를 묻고 사람이 검증한다.
7. 다음 회차에서 AI를 활용해 Pull Request를 만들 준비를 한다.

핵심 내용

코드가 공개되어 있다고 해서 곧바로 오픈소스가 되는 것은 아니다. 사용, 수정, 배포, 기여를 허용하는 조건이 명확해야 한다. 그 조건의 첫 페이지가 LICENSE다.

하지만 LICENSE만으로는 저장소가 운영되지 않는다. 누가 기여할 수 있는지, 어떤 방식으로 Issue와 Pull Request를 만들지, 보안 문제는 어디로 신고할지, 커뮤니티에서 어떤 행동을 기대하는지 함께 정리되어야 한다.

이 회차의 핵심은 라이선스 법률 지식을 외우는 것이 아니라, 저장소를 외부 기여자가 들어올 수 있는 신뢰 구조로 만드는 것이다.

주요 라이선스는 다음 관점으로 이해한다.

- MIT는 짧고 관대한 허용형 라이선스로, 누구나 쉽게 쓰고 fork하기를 원하는 작은 프로젝트에 자주 쓰인다.
- Apache-2.0은 특허 조항을 포함한 허용형 라이선스로, 기업이나 외부 사용 가능성을 조금 더 의식할 때 고려한다.
- GPL 계열은 파생물도 같은 라이선스로 공개하도록 요구하는 카피레프트 성격이 강하다.
- AGPL은 네트워크 서비스 형태까지 고려하는 강한 카피레프트 라이선스다.
- LGPL과 MPL은 라이브러리 사용이나 파일 단위 공개 조건처럼 중간적 성격을 가진다.

AI 프로젝트에서는 공개 범위가 더 복잡해진다. 코드가 공개되어도 데이터셋, 학습 코드, 모델 weights, 평가 코드, 문서는 각각 다른 조건일 수 있다. 따라서 "오픈"이라는 마케팅 표현과 OSI 기준의 오픈소스 여부를 구분해야 한다.

활동

오픈과 오픈소스 구분

여러 프로젝트나 모델 이름을 보고 "오픈소스인지", "사용은 가능하지만 오픈소스라고 단정하기 어려운지", "판단 보류인지"를 구분한다. 이 활동의 목적은 정답 암기가 아니라 이름과 라이선스를 분리해서 보는 습관을 만드는 것이다.

운영 신뢰 구조 만들기

팀 저장소에 다음 문서를 추가하거나 보강한다.

- LICENSE
- CONTRIBUTING.md
- CODE_OF_CONDUCT.md
- SECURITY.md
- Issue Template
- Pull Request Template

CONTRIBUTING에는 최소한 다음 내용을 포함한다.

- 개발 환경 또는 문서 수정 방법
- Issue 작성 기준
- Branch와 Commit 규칙
- Pull Request 작성 기준
- Review와 Merge 기준
- 라이선스 정책
- AI 사용 로그 제출 방식

Pull Request Template에는 다음 내용을 포함한다.

- 무엇을 바꿨는가
- 왜 바꿨는가
- 어떻게 확인했는가

- AI를 사용했다면 무엇을 맡겼는가
- 사람이 검증한 것은 무엇인가
- 외부 코드나 의존성 라이선스를 확인했는가

외부 기여를 시도하는 팀은 외부 저장소의 문서를 복사하지 않는다. 해당 저장소의 규칙을 읽고, 팀 분석 저장소에 기여 준비 메모를 작성한다.

AI 라이선스 리스크 검토

팀이 선택한 라이선스와 프로젝트 설명을 AI에게 제공하고 리스크를 묻는다.

None

우리는 [라이선스 이름]을 선택했고, 프로젝트는 [한 줄 설명]이다.
6개월 뒤 발생할 수 있는 라이선스, 의존성, AI 생성 코드 리스크 3개를 제안해줘.
각 리스크마다 사람이 확인해야 할 공식 출처, 대응 정책, CONTRIBUTING에 넣을
문장을 제안해줘.
법을 조언처럼 단정하지 말고 교육용 점검 항목으로 작성해줘.

AI 답변을 받은 뒤 다음을 확인한다.

- 라이선스 원문 또는 OSI 라이선스 목록과 맞는가
- 팀의 실제 배포 방식과 관련 있는가
- 의존성 라이선스와 충돌 가능성이 있는가
- 문장이 법률 자문처럼 단정적으로 보이지 않는가

검증한 내용은 CONTRIBUTING의 라이선스 정책 섹션에 반영한다.

멘토링

멘토링에서는 팀 저장소의 운영 문서와 라이선스 선택 근거를 확인한다.

확인할 질문은 다음과 같다.

- LICENSE가 존재하고 라이선스 이름이 명확한가?
- CONTRIBUTING에 기여 절차와 라이선스 정책이 있는가?
- SECURITY에 보안 문제 신고 경로가 있는가?
- Pull Request Template에 AI 사용 로그와 라이선스 출처 확인 항목이 있는가?

- AI 라이선스 리스크 답변을 사람이 검증한 흔적이 있는가?
- 4회차에 AI로 분석할 Issue 또는 Pull Request 후보가 정해졌는가?

AI 활용

AI는 라이선스 차이 요약, CONTRIBUTING 초안, 템플릿 초안, 리스크 시나리오 제안에 사용할 수 있다. 사람이 확인해야 할 것은 라이선스 원문, OSI 승인 여부, 팀의 배포 방식, 의존성 정책, 실제 저장소 상태다.

AI 사용 로그에는 다음을 문장으로 남긴다.

- 어떤 라이선스 리스크를 물었는가
- AI가 어떤 리스크를 제안했는가
- 어떤 공식 출처나 원문으로 확인했는가
- CONTRIBUTING에 어떤 문장을 반영했는가

과제

팀 과제는 운영 신뢰 구조 문서를 Pull Request로 제출하는 것이다. 최소한 LICENSE, CONTRIBUTING, Pull Request Template은 포함해야 한다. 가능하면 CODE_OF_CONDUCT, SECURITY, Issue Template까지 포함한다.

개인 과제는 팀 프로젝트에서 사람이 반드시 검증해야 할 라이선스 또는 운영 리스크 두 가지를 작성하는 것이다.

제출 증거는 운영 문서 Pull Request URL, 라이선스 선택 메모, AI 리스크 검토 로그, CONTRIBUTING의 라이선스 정책 섹션이다.

참고 출처

- OSI Licenses: <https://opensource.org/licenses>
- Open Source AI Definition: <https://opensource.org/ai/open-source-ai-definition>
- Choose a License: <https://choosealicense.com/>

AI를 활용한 오픈소스 기여와 운영 자동화

- 일시: 2026-05-21 목 19:00-21:00
- 주제: AI 기반 저장소 이해, 이슈 분석, Pull Request 작성, README와 changelog 보강, RAG/MCP/A2A 개요
- 핵심 메시지: AI는 코드를 대신 써주는 도구가 아니라 오픈소스 기여의 진입장벽을 낮추는 기여 보조 시스템이다.
- 목표 산출물: Pull Request 초안, AI 사용 로그 5단계, 테스트·문서·리뷰 체크리스트, RAG/MCP/A2A 한 줄 정의 노트
- 트랙: 외부 오픈소스 기여와 자체 프로젝트 운영으로 나뉘어 활동한다.

학습 목표

수강생은 이 회차 후 다음을 수행할 수 있어야 한다.

1. 낯선 저장소를 AI 없이 분석한 결과와 AI와 함께 분석한 결과를 비교한다.
2. 이슈 분석, 재현, 수정 범위 산정, Pull Request 본문 작성, 리뷰 대응 단계에서 AI와 사람의 역할을 분리한다.
3. 외부 오픈소스 기여 Pull Request 초안 또는 자체 프로젝트 기능·문서 Pull Request 초안을 만든다.
4. README 또는 changelog를 AI로 보강하되 실제 변경 내용과 일치하는지 사람이 검증한다.
5. RAG, MCP, A2A의 개념적 위치를 한 줄씩 설명한다.

수업 전 준비

수강생은 3회차에서 만든 LICENSE, CONTRIBUTING, Pull Request Template을 확인한다. 작업할 Issue 하나를 고르고, AI 도구에 로그인해 둔다.

외부 오픈소스 기여 팀은 대상 저장소의 CONTRIBUTING과 LICENSE를 다시 읽는다. 자체 프로젝트 팀은 팀 저장소에서 이번 회차에 처리할 작은 Issue를 하나 고른다.

수업 흐름

1. 3회차 운영 문서가 AI 활용의 안전장치가 된다는 점을 확인한다.
2. 낫선 저장소를 AI 없이 한 번, AI와 함께 한 번 분석한다.
3. AI 사용 로그 5단계 형식을 확인한다.
4. 이슈 분석부터 Pull Request 작성까지 AI와 사람의 역할을 분리한다.
5. 트랙별로 Pull Request 초안을 만든다.
6. RAG, MCP, A2A를 한 줄 정의로 정리한다.
7. README 또는 changelog를 보강한다.
8. 다음 회차의 멀티 에이전트 작업 후보를 정한다.

핵심 내용

AI는 저장소 구조를 빠르게 요약하고, 관련 파일 후보를 제안하고, Pull Request 본문 초안을 만드는 데 도움이 된다. 그러나 AI 결과는 초안이다. 실제 실행, 테스트, diff 확인, 보안, 개인정보, 라이선스 검토는 사람이 책임진다.

AI 사용은 다음 5단계로 기록한다.

1. 이슈 분석: AI는 문제 요약과 원인 후보를 제안할 수 있다. 사람은 이슈 원문과 프로젝트 규칙을 확인한다.
2. 재현: AI는 재현 절차 초안을 만들 수 있다. 사람은 실제로 실행하고 실패 로그를 확인한다.
3. 수정 범위 산정: AI는 변경 파일 후보를 제안할 수 있다. 사람은 범위가 너무 크지 않은지 판단한다.
4. Pull Request 본문 작성: AI는 본문 초안을 만들 수 있다. 사람은 실제 diff와 일치하는지 확인한다.
5. 리뷰 대응: AI는 답변 초안을 만들 수 있다. 사람은 메인테이너 의도와 실제 수정 여부를 확인한다.

RAG는 외부 검색이나 문서를 LLM 답변에 결합하는 접근이다. README와 Issue를 AI에게 함께 제공하는 활동은 가장 단순한 형태의 RAG로 볼 수 있다.

MCP는 LLM 애플리케이션이 외부 데이터와 도구에 표준 방식으로 연결되도록 하는 프로토콜이다. GitHub, 파일, 데이터베이스, API 같은 도구 연결을 이해하는 배경이 된다.

A2A는 서로 다른 AI 에이전트가 통신하고 협업하도록 하는 프로토콜이다. 5회차에서 Planner, Coder, Reviewer, Tester, Documenter 역할을 나누는 흐름과 연결된다.

활동

낮선 저장소 분석 비교

먼저 AI 없이 README, Issues, Pull Requests, LICENSE를 보고 저장소 구조와 기여 가능성을 판단한다. 이후 같은 저장소 정보를 AI에게 제공하고 요약을 받는다.

비교할 내용은 다음과 같다.

- AI가 빠르게 찾아준 정보는 무엇인가
- 사람이 더 정확하게 확인한 정보는 무엇인가
- AI가 추정하거나 틀릴 수 있는 부분은 무엇인가
- 기여 후보로 삼아도 되는 작업은 무엇인가

외부 오픈소스 기여 Pull Request 초안

외부 기여 팀은 작은 Issue를 하나 고른다. **good first issue**, **docs**, **help wanted** 같은 라벨이 붙은 작업이 좋다.

진행 순서는 다음과 같다.

1. Issue 원문을 읽는다.
2. CONTRIBUTING과 LICENSE를 확인한다.
3. AI에게 이슈 요약과 수정 후보를 요청한다.
4. 사람이 가장 작은 Pull Request 범위를 고른다.
5. Fork, Branch, Commit, Pull Request 초안을 만든다.
6. Pull Request 본문에 AI 사용 로그를 작성한다.

자체 프로젝트 기능 또는 문서 Pull Request

자체 프로젝트 팀은 팀 저장소의 Issue 하나를 고른다.

진행 순서는 다음과 같다.

1. Issue의 완료 조건을 확인한다.
2. AI로 파일 구조와 영향 범위를 분석한다.
3. 변경 범위를 작은 기능 하나 또는 문서 개선 하나로 줄인다.
4. 테스트 또는 문서 체크리스트를 작성한다.
5. Pull Request 초안을 만든다.
6. AI 사용 로그와 사람 검증 기록을 작성한다.

README 또는 changelog 보강

Pull Request 변경 내용을 바탕으로 README 또는 changelog를 보강한다.

AI에게 요청할 수 있는 예시는 다음과 같다.

None

다음 Pull Request 변경 내용과 현재 README를 보고 README 보강안을 작성해줘.
실제 변경에 없는 기능은 쓰지 말고, 설치, 사용, 검증 방법 중 빠진 항목만 제안해줘.
마지막에 사람이 검증해야 할 항목을 붙여줘.

AI가 만든 문서는 실제 변경과 맞는 부분만 반영한다.

멘토링

멘토링에서는 Pull Request 초안, AI 사용 로그, 검증 체크리스트를 확인한다.

확인할 질문은 다음과 같다.

- Pull Request 초안이 실제 링크로 존재하는가?
- 변경 범위가 작고 리뷰 가능한가?
- AI 사용 로그 5단계가 비어 있지 않은가?
- 사람이 검증한 항목이 구체적인가?
- 테스트 또는 문서 증거가 있는가?
- RAG, MCP, A2A 한 줄 정의가 남아 있는가?
- 5회차에 멀티 에이전트로 처리할 Issue가 정해졌는가?

AI 활용

AI는 코드 설명, 원인 후보, 테스트 초안, Pull Request 본문, README 또는 changelog 초안을 만드는 데 사용할 수 있다. 사람이 확인해야 할 것은 실행 결과, 테스트 통과 여부, 실제 diff, 기존 스타일, 보안, 개인정보, 라이선스다.

AI 사용 로그에는 다음을 문장으로 남긴다.

- 이슈 분석에서 AI가 제안한 것
- 재현 과정에서 사람이 실행한 것
- 수정 범위를 줄인 기준
- Pull Request 본문에서 사람이 고친 부분
- 리뷰 대응이나 문서 보강에서 검증한 내용

과제

팀 과제는 외부 기여 Pull Request 초안 또는 자체 프로젝트 기능·문서 Pull Request 초안을 제출하는 것이다. Pull Request 본문이나 별도 문서에 AI 사용 로그 5단계를 남긴다.

추가로 README 또는 changelog 보강 결과를 제출한다. 개인 과제는 RAG, MCP, A2A 한 줄 정의와 AI 사용 회고를 남기는 것이다.

제출 증거는 Pull Request URL, AI 사용 로그, 테스트 또는 문서 증거, RAG/MCP/A2A 한 줄 정의다.

참고 출처

- RAG 논문: <https://arxiv.org/abs/2005.11401>
- Model Context Protocol: <https://modelcontextprotocol.io/docs/getting-started/intro>
- A2A Protocol: <https://a2a-protocol.org/latest/>

AI 멀티 에이전트로 오픈소스 프로젝트 개발

- 일시: 2026-05-28 목 19:00-21:00
- 주제: Planner, Coder, Reviewer, Tester, Documenter 역할 분리와 Pull Request 완성
- 핵심 메시지: AI를 잘 쓰는 개발자는 하나의 챗봇에게 질문하는 사람이 아니라 역할이 나뉜 AI 작업 흐름을 설계하는 사람이다.
- 목표 산출물: 멀티 에이전트 흐름으로 만든 Pull Request, 테스트 또는 데모, 문서 업데이트, 역할별 AI 사용 기록, 최종 발표 초안 5장, 5분 측정 결과
- 트랙: 외부 오픈소스 기여와 자체 프로젝트 운영으로 나뉘어 활동한다.

학습 목표

수강생은 이 회차 후 다음을 수행할 수 있어야 한다.

1. Planner, Coder, Reviewer, Tester, Documenter 역할을 실제 작업 흐름에 매핑한다.
2. MCP는 도구 연결, A2A는 에이전트 협업이라는 차이를 사례로 설명한다.
3. hallucination, 보안, 라이선스, 의존성 검증의 안전 장치를 Pull Request에 적용한다.
4. 외부 오픈소스 기여 또는 자체 프로젝트에서 작은 Pull Request를 멀티 에이전트 흐름으로 완성한다.
5. 6회차 발표 초안을 5장으로 만들고 5분 안에 말해 본다.

수업 전 준비

수강생은 4회차 Pull Request 초안과 AI 사용 로그를 다시 읽는다. 한 명의 AI에게 너무 많은 일을 맡겨 결과가 흐려졌던 지점을 하나 찾는다.

팀은 이번 회차에 끝까지 처리할 작은 Issue 하나를 고른다. 또한 6회차 발표에 사용할 공개 증거 링크를 모아 둔다.

수업 흐름

1. 4회차에서 AI를 한 명의 도구처럼 썼을 때의 한계를 돌아본다.
2. 하나의 작업을 다섯 역할로 나누는 이유를 이해한다.
3. MCP와 A2A가 역할 분리 흐름과 어떻게 연결되는지 정리한다.
4. 트랙별로 작은 Issue 하나를 멀티 에이전트 흐름으로 처리한다.
5. 안전 장치 네 가지를 Pull Request에 적용한다.
6. 발표 초안 5장을 만들고 5분 안에 말해 본다.
7. 다음 회차 발표와 포트폴리오 정리 준비 항목을 확정한다.

핵심 내용

4회차에서는 AI를 기여 보조 시스템으로 사용했다. 그러나 하나의 AI에게 이슈 분석, 구현, 테스트, 리뷰, 문서화를 모두 맡기면 결과가 흐려질 수 있다. 5회차에서는 작업을 역할별로 나누고, 사람은 전체 흐름의 책임자로 검증한다.

Planner는 Issue와 README를 보고 작업을 작은 카드로 나눈다. Coder는 작업 카드 하나를 코드나 문서 변경으로 구현한다. Tester는 변경이 실제로 동작하는지 확인한다. Reviewer는 hallucination, 보안, 라이선스, 의존성, 스타일을 점검한다. Documenter는 README, changelog, Pull Request 본문이 실제 변경과 맞도록 정리한다.

MCP는 AI가 파일, GitHub, 데이터베이스, API 같은 외부 도구에 연결되는 방식과 관련된다. A2A는 여러 에이전트가 서로 역할을 나누고 결과를 주고받는 방식과 관련된다. 이 수업에서 중요한 것은 프로토콜 구현이 아니라 역할 분리와 검증 책임이다.

안전 장치는 네 가지다. AI가 존재하지 않는 API나 파일을 말하면 머지하지 않는다. 비밀키, 개인정보, 과도한 권한이 보이면 머지하지 않는다. 출처가 불명확한 코드나 호환성 불명 의존성이 있으면 머지하지 않는다. 불필요하거나 관리되지 않는 패키지를 추가하지 않는다.

활동

역할 분리 설계

팀은 선택한 Issue 하나를 다섯 역할로 나눈다.

Planner가 할 일은 문제를 작은 작업 카드로 나누는 것이다. 작업 카드는 완료 조건, 예상 변경 파일, 테스트 방법, 리스크를 포함한다.

Coder가 할 일은 작업 카드 하나만 구현하는 것이다. 범위를 넘기지 않는 것이 중요하다.

Tester가 할 일은 실제 실행 또는 검증 결과를 남기는 것이다. 테스트가 없다면 최소한 재현 절차나 수동 검증 결과를 남긴다.

Reviewer가 할 일은 변경 사항을 검토하는 것이다. 특히 hallucination, 보안, 라이선스, 의존성을 확인한다.

Documenter가 할 일은 README, changelog, Pull Request 본문을 실제 변경과 맞게 갱신하는 것이다.

외부 오픈소스 기여 흐름

외부 오픈소스 기여 팀은 대상 저장소의 Issue를 작은 작업 카드로 쪼갬다. fork 저장소에서 하나의 카드만 구현하고, 대상 프로젝트의 CONTRIBUTING 기준에 맞춰 테스트와 문서를 정리한다.

외부 프로젝트의 응답이 늦다면 그것도 오픈소스 현실의 일부로 기록한다. 이 경우 팀 분석 저장소나 자체 보조 저장소에서 같은 멀티 에이전트 흐름을 시연하고, 6회차 발표에서 외부 응답 지연과 대체 전략을 설명한다.

자체 프로젝트 운영 흐름

자체 프로젝트 팀은 팀 저장소의 Issue 하나를 골라 작업 카드로 나눈다. 코드 변경이 어려우면 문서 개선이나 테스트 도입도 가능하다.

Reviewer 역할은 팀원 한 명과 AI를 함께 사용할 수 있다. 단, 최종 책임은 팀에게 있다. Documenter는 변경 내용을 README, changelog, Pull Request 본문에 반영한다.

역할별 AI 사용 기록

역할별로 다음 내용을 문장으로 남긴다.

- 역할 이름
- AI에게 맡긴 일
- AI가 만든 결과
- 사람이 확인하거나 수정한 내용
- 최종 Pull Request에 반영된 내용

예시는 다음과 같다.

None

역할별 AI 사용 기록

Planner

- 맡긴 일:
- 결과:
- 사람이 확인한 것:

Coder

- 맡긴 일:
- 결과:
- 사람이 확인한 것:

Tester

- 맡긴 일:
- 결과:
- 사람이 확인한 것:

Reviewer

- 맡긴 일:
- 결과:
- 사람이 확인한 것:

Documenter

- 맡긴 일:
- 결과:
- 사람이 확인한 것:

발표 초안 5장 만들기

6회차 발표는 5분 발표와 5분 질의응답으로 진행된다. 5회차에서는 발표 초안을 5장으로 압축하고 한 번 시간을 측정한다.

1장은 선택한 문제와 트랙을 설명한다. 2장은 6주 동안 만든 Issue, Pull Request, Commit을 보여준다. 3장은 AI 활용과 사람이 검증한 부분을 보여준다. 4장은 라이선스, 운영 정책, 로드맵을 정리한다. 5장은 배운 점과 다음 기여 약속을 담는다.

발표를 해 본 뒤 실제 발표 시간, 5분 초과 원인, 잘라낼 내용, 추가해야 할 공개 링크를 기록한다.

멘토링

멘토링에서는 Pull Request, 역할별 AI 사용 기록, 테스트 또는 데모, 발표 초안을 확인한다.

확인할 질문은 다음과 같다.

- 다섯 역할이 모두 사용되었거나 최소한 시뮬레이션되었는가?
- Reviewer가 hallucination, 보안, 라이선스, 의존성을 확인했는가?
- Tester가 실제 실행 또는 검증 결과를 남겼는가?
- Documenter가 README, changelog, Pull Request 본문 중 하나를 갱신했는가?
- 발표 초안 5장이 있는가?
- 발표가 5분 안에 끝나는가?
- 6회차 발표 8개 항목이 빠지지 않았는가?

AI 활용

AI는 작업 카드 작성, 코드 또는 문서 초안, 테스트 케이스, 리뷰 코멘트, 발표 구조를 만드는 데 사용할 수 있다. 사람이 확인해야 할 것은 역할 간 모순, 실제 실행, 테스트 의미, 문서와 기능의 일치, 기여 범위 과장 여부다.

역할별 AI 사용 기록에는 프롬프트 전문을 모두 붙일 필요는 없다. 대신 어떤 역할에 무엇을 맡겼고, 사람이 무엇을 검증했는지 명확히 남긴다.

과제

팀 과제는 멀티 에이전트 흐름으로 만든 Pull Request 하나를 제출하는 것이다. 코드 기능이 아니어도 문서 또는 테스트 Pull Request가 가능하다. 테스트 또는 데모 증거와 역할별 AI 사용 기록도 함께 제출한다.

추가로 발표 슬라이드 5장 초안과 5분 측정 결과를 제출한다. 개인 과제는 자신이 맡은 역할과 사람이 검증한 내용을 회고하는 것이다.

제출 증거는 Pull Request URL, 실행 또는 테스트 결과, 문서 변경, 역할별 AI 사용 기록, 발표 초안이다.

참고 출처

- Model Context Protocol: <https://modelcontextprotocol.io/docs/getting-started/intro>
- A2A Protocol: <https://a2a-protocol.org/latest/>

